Welcome to your next lesson: Lists ii: indexing and slicing.

I'm preparing your personalized lesson based on what you've already learned, what's coming up next, and the best ways to help you grow.

Step by step, you’re moving forward.

Ron, let’s build on what you did in the last lesson.

Imagine a short playlist in Python:

Python

```
songs = ["Intro", "First Song", "Second Song", "Finale"]
```

Each item has a position (index). Python starts counting at 0, not 1:

Index	Element
0	"Intro"
1	"First Song"
2	"Second Song"
3	"Finale"

So:

```
songs[0] → "Intro"
songs[1] → "First Song"
```

This is called zero-based indexing.

Does this 0–3 index layout make sense so far, or does any part of it feel off or confusing?

It makes sense

What do you think nums[2] is in that question? 👇

Given this list:

```
nums = [10, 20, 30, 40]
```

What does nums[2] give you?

A.

10

B.

20

C.

30

D.

40

Nice, exactly right 🎯

With:

Python

```
nums = [10, 20, 30, 40]
# indexes:   0   1   2   3
```

Index 2 is the third slot in the list
The value in that slot is 30
So nums[2] gives you 30.
Next small step: want to see how negative indexes let you grab items from the end of the list (like nums[-1])?

Yes, show me negative indexes

Awesome, let’s go step by step with negative indexes 👇

Think of the same list:

Python

```
nums = [10, 20, 30, 40]
# + indexes:  0   1   2   3
# - indexes: -4  -3  -2  -1
```

So from the end:

nums[-1] → 40 (last item)
nums[-2] → 30 (second from the end)

It’s like counting backward from the right side of the list.

Quick check:

Without running code, what do you expect nums[-3] to be in that list?

20

Exactly right again 🎯

Using:

Python

```
nums = [10, 20, 30, 40]
# + indexes:  0   1   2   3
# - indexes: -4  -3  -2  -1
```

nums[-3] lines up with index 1
Index 1 holds the value 20
So nums[-3] → 20. Perfect.
Next, I want to show how this works with slices (sub-lists), like taking the middle part of a list.

Are you okay to move on to slicing now?

Yes, let's try slicing

Great 👍 let’s start simple.

Take this list:

Python

```
nums = [10, 20, 30, 40, 50]
```

indexes: 0   1   2   3   4
A slice takes a range of positions:

Python

middle = nums[1:4]
This means:

start at index 1 (included)
go up to index 4 (but stop before 4)

So it grabs indexes 1, 2, 3 → [20, 30, 40].

In general: list[start:stop] → from start up to (but not including) stop.

Does that rule — "start included, stop excluded" — feel clear so far?

Yes, it's clear

Nice 😄 that's an important rule for slicing.

Let's use it on a new list:

Python

letters = ["a", "b", "c", "d", "e", "f"]
indexes: 0   1   2   3   4   5
Look at this slice:

Python

part = letters[2:5]
Step by step:

start at index 2 → "c"
go up to (but not including) index 5 → so we take indexes 2, 3, 4

So part will be a new list with 3 elements.

What do you think part will contain? You can just tell me the list in order.

"c" "d" and "e"

Perfect ☑

You applied the rule exactly right:

Python

letters = ["a", "b", "c", "d", "e", "f"]
part = letters[2:5]
index 2 → "c"
index 3 → "d"
index 4 → "e"
index 5 → stop (don't include)

So part is:

Python

["c", "d", "e"]
Next small idea: you can omit one side:

letters[:3] → from start up to index 3 (0,1,2)
letters[3:] → from index 3 to the end

Want to try guessing what letters[:3] gives?

"a", "b" and "c" number 1
"d" "e" and "f" second one

You nailed both of those 👏

Given:

Python

```
letters = ["a", "b", "c", "d", "e", "f"]
letters[:3] → start of list up to index 3 (not included)
→ ["a", "b", "c"] ☑

letters[3:] → from index 3 to the end
→ ["d", "e", "f"] ☑
```


So you've got:

basic indexing
negative indexing
and core slicing (including open-ended) all correct so far 🎯
Next useful trick: you can also make a copy of the whole list with slicing:

Python

```
copy_letters = letters[:]
```
This creates a new list with the same elements.

Want to see a quick example of why copying like this can matter, or do you want to practice a bit more with slices first?

I'd like an example

Nice, let's do a clear example step by step 🧩

Start with a list:

Python

```
numbers = [1, 2, 3]
copy_numbers = numbers[:]   # make a copy
```
Now change the original:

Python

```
numbers[0] = 99
```
After this:

numbers is now [99, 2, 3]
copy_numbers is still [1, 2, 3] (unchanged)

So [:] gave you a separate list. Changing one doesn't change the other.

Does that difference between numbers and copy_numbers make sense so far?

Yes, that makes sense

Great 👍

Let’s look at one more key skill: changing a list item using its index.

Step by step:

Python

```
colors = ["red", "green", "blue"]
# indexes:     0       1        2

colors[1] = "yellow"
```

Here’s what happens:

index 1 currently holds "green"
colors[1] = "yellow" replaces "green" with "yellow"

After that line, the list becomes:

Python

```
["red", "yellow", "blue"]
```

You’re not making a new list — you’re updating the existing one in place.

Quick check: if we start again with

Python

```
colors = ["red", "green", "blue"]
colors[2] = "purple"
```

What will colors look like after this change?

"red", "green", "purple"

Exactly ☑

Starting from:

Python

```
colors = ["red", "green", "blue"]
colors[2] = "purple"
```

index 2 was "blue"
we replace it with "purple"

So the final list is:

Python

```
["red", "green", "purple"]
```

You’ve got indexing and in-place replacement down 😄

Next important piece for this lesson is what happens when you use an index that’s too big and you get an IndexError. That’s key for debugging.

Are you ready to look at what an IndexError is and how to avoid it?

Yes, let's see it

Nice, we’ll keep this one very concrete 👇

Take this list:

Python

nums = [10, 20, 30]
indexes: 0   1   2
Now look at this line:

Python

print(nums[3])
Valid indexes are 0, 1, 2
3 is past the end of the list
So Python raises an error:
Text

IndexError: list index out of range
This means: “You tried to access a position that doesn’t exist in this list.”

Just to be sure: does “valid indexes are 0 to len(list) - 1” make sense with this example?

Yes, that makes sense

What do you think? Take your time with the question below 👇

Given this list:

items = [5, 6, 7]
Which index will cause an IndexError if you try to access items[?]?

A.

0

B.

1

C.

2

D.

3

Nice work Ron 🎯 You spotted the error case exactly.

With:

Python

items = [5, 6, 7]
indexes: 0 1 2
Valid indexes are 0, 1, 2
items[3] is past the end → there is no index 3
So Python raises: IndexError: list index out of range

That’s exactly the pattern to watch for.

You’ve now worked with:

positive & negative indexing
slicing (including open-ended)
replacing elements by index
noticing when an index is out of range

Want to do one small “real-world” practice with a list to tie it all together, or do you

feel ready to wrap up this lesson?

I feel ready to finish

Nice work today Ron 🎓

Here’s what you can now do with lists in Python:

Use indexes (positive and negative) to grab single items, like nums[0] or nums[-1]
Use slices like nums[1:4], nums[:3], nums[3:], and nums[:] to create sublists or full copies
Replace items in place, like colors[1] = "yellow"
Recognize and avoid IndexError: list index out of range by remembering valid indexes go from 0 to len(list) - 1
You’re thinking about indexes and slices the right way — that will pay off a lot when we start working with loops and more complex data later on 🚀

When you’re ready, you can jump into the next lesson from the Learn tab.
If anything about indexing or slicing feels fuzzy later, come back and I’ll walk through it with you again step by step.

How would you rate this lesson?